F1 Insight API — Technical Report

COMP3011 — Web Services and Web Data | | March 2026

GitHub: [Insert Link] | Live API: https://f1-insight-api.onrender.com/docs | Slides: [Insert Link]

1. Design and Architectural Choices

1.1 Technology Stack

Layer	Technology	Justification
Framework	Python 3.12 + FastAPI 0.115	Auto-generated OpenAPI docs; Pydantic validation; Depends() injection
Database	PostgreSQL 14	Relational dataset; multi-table JOIN/GROUP BY for analytics
ORM	SQLAlchemy 2.0	Declarative models; type-safe 2.0 API; Base.metadata.create_all()
Auth	JWT (python-jose + passlib[bcrypt])	Stateless RESTful tokens; role-based access (user/admin)
Testing	pytest + httpx + SQLite in-memory	TestClient integration tests; dependency override for isolation

1.2 Justification of Key Choices

Why FastAPI over Django. FastAPI was chosen over Django REST Framework (the framework taught in COMP3011) for three reasons: (1) automatic OpenAPI/Swagger documentation generation from type hints, which directly satisfies the API documentation deliverable without additional configuration; (2) built-in request validation via Pydantic, eliminating manual input checking; and (3) native dependency injection via Depends(), which enabled clean separation of database sessions and authentication logic. Choosing a non-default framework demonstrates independent technical judgement, which the marking criteria values at the 70+ band.

Why PostgreSQL over NoSQL. The Ergast F1 dataset is inherently relational — results reference races, drivers, constructors, and status via foreign keys. The analytics endpoints rely heavily on multi-table JOIN and GROUP BY aggregations (e.g., computing career statistics across results, races, and drivers). A document store such as MongoDB would have required either data denormalisation — introducing update anomalies across 26,000+ results — or application-level joins that would negate database engine benefits.

Why JWT over sessions. Stateless token-based authentication aligns with RESTful principles and integrates naturally with FastAPI's OAuth2 support. Role-based access control distinguishes regular users (create/update) from admins (delete), reflecting a realistic multi-tenant API rather than a single all-powerful user model.

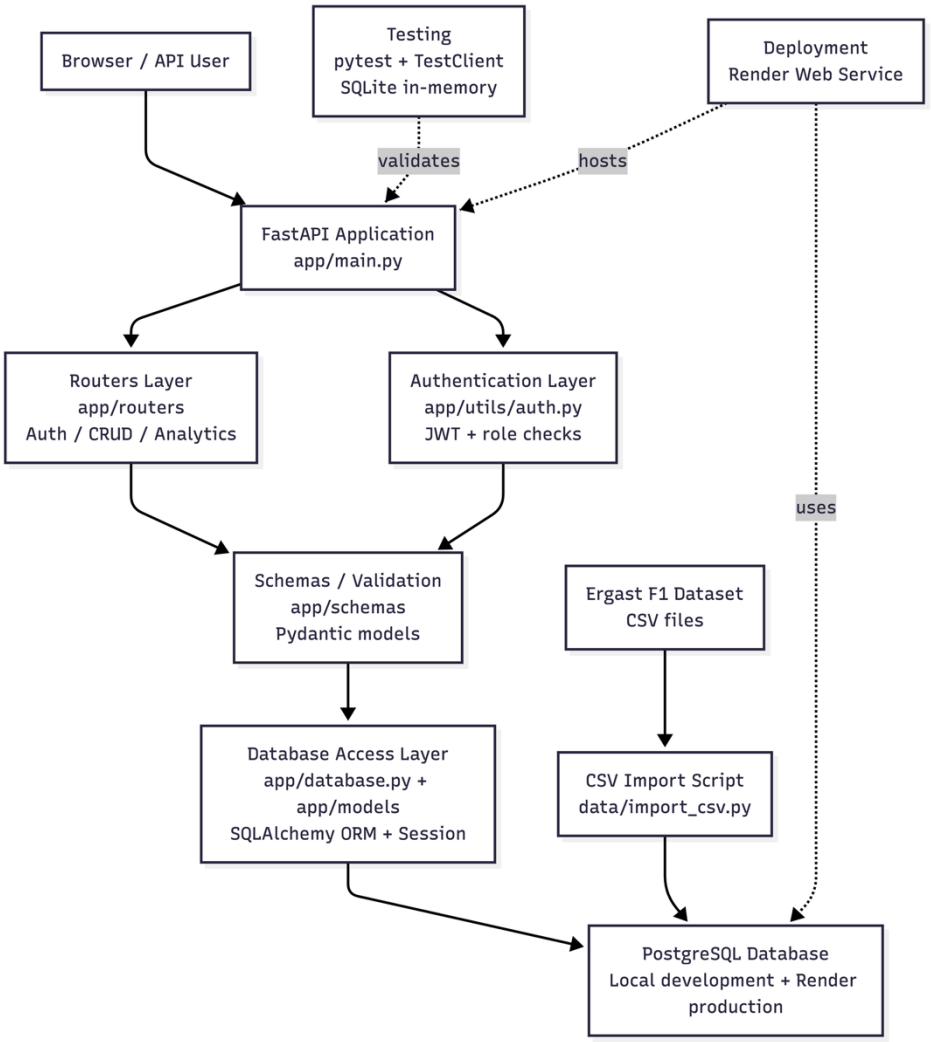

Figure 1 shows the high-level architecture of the system. Client requests enter the FastAPI application through the router layer, are validated using Pydantic schemas, authenticated through JWT-based dependencies where required, and are then processed against PostgreSQL via SQLAlchemy models and sessions. Historical Formula 1 data is imported separately through a dedicated CSV ingestion script. Testing is performed with pytest and FastAPI TestClient using an isolated SQLite in-memory database, while deployment is handled through Render with a managed PostgreSQL instance.

1.3 API Design Highlights

The API exposes 6 router groups under /api/v1/: Authentication, Drivers, Circuits, Constructors, Analytics, and Advanced Analytics. Full CRUD is supported for 3 core entities; analytics endpoints are read-only and compute all statistics from raw race data — no pre-aggregated standings tables are used.

Multi-dimensional sorting. List endpoints support a sort_by parameter with five modes: popularity (all-time points), recent (latest season points), wins (career wins), newest, and oldest. This was inspired by content recommendation patterns in video streaming platforms and iteratively developed from a simple asc/desc toggle into a 5-option system performing real-time aggregation across drivers, results, and races via outerjoin subqueries. I chose single-season data for the recent metric after analysing 1, 3, and 5-season alternatives — single-season maximises differentiation from all-time popularity (e.g., Schumacher ranks #2 in popularity but disappears from recent), whereas a multi-year window would overlap too heavily.

Win probability model. Combines four weighted factors: circuit history (35%), recent form over last 5 races (30%), grid-to-finish conversion (20%), and career baseline (15%). Prioritised first among four advanced analytics options because the coursework brief explicitly mentions "win probabilities." The response includes full factor breakdowns for transparency.

Performance summary. Scores drivers out of 100 across five dimensions (qualifying pace, race pace, consistency, overtaking, pit stop efficiency), each drawing from different tables to demonstrate cross-table

analytical capability.

2. Data Source

The API uses the Ergast Motor Racing Developer API dataset (Kaggle mirror), covering F1 seasons 1950-2024. The Ergast API was retired in 2024; this project imports static CSV data into a self-hosted PostgreSQL database rather than depending on any third-party live API — ensuring data autonomy. Ten CSV files are imported:

CSV File	Records	CSV File	Records
seasons.csv	75	races.csv	1,125
circuits.csv	77	results.csv	26,000+
drivers.csv	859	qualifying.csv	10,000+
constructors.csv	212	pit_stops.csv	10,000+
status.csv	139	lap_times.csv	550,000+

Together these total approximately 670,000 records across 850+ drivers, 210+ constructors, and 77 circuits. Four additional files (driver_standings, constructor_standings, constructor_results, sprint_results) were intentionally excluded as their data is computed dynamically by analytics endpoints, avoiding redundancy and demonstrating computational capability. The database schema contains 11 tables (10 data tables plus users for authentication) with a star/network topology where the results table simultaneously references four dimension tables (races, drivers, constructors, status).

3. Challenges, Testing, and Reflection

3.1 Technical Challenges

N+1 query problem. The compare_drivers and pit_stop_analysis endpoints initially issued a separate query for each row's constructor name and race details — generating hundreds of queries when comparing drivers with 300+ races. Fixed by replacing per-row lookups with bulk .in_() fetches and Python dict lookups, reducing query count from O(n) to O(1) per endpoint call.

Pit stop outlier filtering. Red-flag periods distorted average pit stop statistics. A 45,000ms threshold was established to filter anomalous stops, and the median was chosen over the mean as pit stop durations exhibit right-skewed distributions where a single outlier significantly distorts the average.

SQLite/PostgreSQL divergence. SQLite disables foreign keys by default and handles NULL sorting differently. Resolved by attaching an event listener for PRAGMA foreign_keys=ON, using StaticPool for shared in-memory connections, and writing ORM-level queries abstracting away dialect differences.

bcrypt incompatibility. passlib's bcrypt backend broke silently with bcrypt>=4.1. Resolved by pinning bcrypt==4.0.1 after tracing through passlib's source code.

3.2 Testing

The API is validated by 66 integration tests across 5 test files, achieving 77% overall code coverage. Tests use FastAPI's TestClient with SQLite in-memory databases via dependency override.

Test File	Tests	Coverage	Focus Area
test_auth.py	11	100%	Registration, login, JWT validation, role-based access
test_drivers.py	15	92%	CRUD, pagination, filtering, sorting, auth guards
test_circuits.py	13	92%	CRUD, aggregated stats, race history, FK constraints
test_constructors.py	12	89%	CRUD, career summaries, top drivers, auth/admin checks
test_analytics.py	10	66%	Career stats, leaderboards, pit stop analysis, circuit history

Key patterns include: a seed_data fixture with a minimal realistic F1 dataset (3 drivers, 3 constructors, 2 circuits, 3 races, 8 results); separate auth_headers/admin_headers fixtures for role-based testing; and autouse fixtures that create/drop all tables between tests for full isolation. The advanced_analytics module has 19% coverage — these multi-factor endpoints require more sophisticated seed data.

3.3 Reflection

The main lesson was the tension between computational expressiveness and query performance: computing analytics from raw data provides flexibility but demands careful query design. The N+1 problem showed that ORM convenience can mask performance issues only visible at scale. The SQLite/PostgreSQL divergence highlighted the need for CI testing against the production engine. The bcrypt issue reinforced the importance of dependency pinning.

4. Limitations and Future Development

Current limitations include: no Alembic migrations (acceptable for a static dataset but not for evolving schemas); low test coverage on advanced analytics (19%); no caching layer for expensive aggregate queries; permissive CORS (allow_origins=["*"]); and Render free-tier cold-start latency (30-50s). Future improvements would include: Alembic migration support, Redis caching for leaderboard queries, a dedicated service layer, Docker containerisation, live data integration via OpenF1 API, and a GraphQL layer to reduce over-fetching.

5. Generative AI Declaration

5.1 Tools and Scope

Claude (Anthropic) was used via the claude.ai web interface throughout this project, in a mix of English and Chinese. No other AI tools were used. Full conversation logs are attached as supplementary material. The table below classifies AI usage by level:

Usage Level	Examples	Band
Architecture exploration	Framework comparison (Django vs FastAPI), database selection (PostgreSQL vs NoSQL), auth strategy (JWT vs sessions), win probability algorithm design (4 approaches compared)	90-100
Critical evaluation	Questioning 10-table complexity, proposing multi-dimensional sorting, analysing single vs multi-season data ranges, rejecting over-engineering	80-89
Concept understanding	Learning CRUD/endpoint/JWT concepts, understanding ORM query patterns, reviewing FastAPI dependency injection	70-79
Code generation	Project skeleton, CSV import script, CRUD routers, analytics queries, test fixtures	50-59
Debugging	N+1 query identification, bcrypt version tracing, SQLite/PostgreSQL divergence fixes, Swagger refresh issue	50-59

5.2 Human-Driven Design Decisions

GenAI served as an implementation accelerator, but the critical design decisions were human-driven:

1. Multi-dimensional sorting concept. I proposed this after observing content recommendation systems in video streaming platforms. Claude and I iteratively developed it from a simple toggle into a 5-option system. The key insight — that single-season data maximises differentiation from all-time rankings — came from my analysis of how retired drivers (Schumacher ranking #2 in popularity but absent from recent) behave under different time windows. I specifically questioned whether 1, 3, or 5 seasons should be used.

2. Feature prioritisation. When presented with four advanced analytics options, I chose Win Probability

first because the coursework brief explicitly mentions "win probabilities" — a deliberate decision to maximise assessment-relevant coverage.

3. Data quality and scope decisions. The pit stop outlier threshold (45,000ms) and median-over-mean choice were suggested through AI-assisted exploration but validated by me against actual data distributions. When I questioned whether the 10-table design was over-engineered, joint analysis concluded that complexity benefits the 80+ criteria ("creative data design") but only 3 resources should have full CRUD — balancing depth with efficiency.

5.3 What I Modified or Rejected

Driver search: Claude's initial search covered only forename/surname. I extended it to driver_ref and code fields, added NULL safety for the code column, and changed default sort to descending — reasoning that searching "Verstappen" should show Max (active) before Jos (retired).

Verification: I cross-referenced SQLAlchemy query patterns against official 2.0 documentation, tested every endpoint via Swagger UI, and validated analytical results against known F1 statistics. The primary risk — accepting generated code without understanding query semantics — was mitigated through systematic review.

References

Ergast Developer API (2024). Motor Racing Data. http://ergast.com/mrd/ — FastAPI (Ramirez, 2018). https://fastapi.tiangolo.com/ — SQLAlchemy (Bayer, 2012). https://www.sqlalchemy.org/ — Pydantic (Colvin, 2017). https://docs.pydantic.dev/ — pandas (McKinney, 2010). https://pandas.pydata.org/ — python-jose (Melvill, 2015). JWT for Python. — passlib (Warren, 2008). Password hashing with bcrypt. — Render (2019). https://render.com/